Create authentication service that returns JWT:

pom.xml:

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-parent</artifactId>

    <version>4.1.0</version>

    <relativePath/> <!-- lookup parent from repository -->

  </parent>

  <groupId>com.cognizant</groupId>

  <artifactId>spring-learn</artifactId>

  <version>0.0.1-SNAPSHOT</version>

  <name/>

  <description/>

  <url/>

  <licenses>

    <license/>

  </licenses>

  <developers>

    <developer/>
```

```
</developers>
<scm>
  <connection/>
  <developerConnection/>
  <tag/>
  <url/>
</scm>
<properties>
  <java.version>25</java.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc-test</artifactId>
    <scope>test</scope>
```

```
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-api</artifactId>
      <version>0.11.5</version>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-impl</artifactId>
      <version>0.11.5</version>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-jackson</artifactId>
      <version>0.11.5</version>
      <scope>runtime</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
```

```
        <plugin>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
      </plugins>
    </build>

</project>
```

SecurityConfig.java:

```
package com.cognizant.spring_learn.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
      .csrf(csrf -> csrf.disable()) // Disable CSRF for REST APIs
```

```
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/authenticate").permitAll() // Allow public access to generation endpoint
            .anyRequest().authenticated()              // Secure all other endpoints
        );

    return http.build();
  }
}
```

AuthController.java:

```
package com.cognizant.spring_learn.controller;

import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.Collections;
import java.util.Date;
import java.util.Map;
import javax.crypto.SecretKey;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;
```

```
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;

@RestController
public class AuthController {

   private static final Logger LOGGER = LoggerFactory.getLogger(AuthController.class);

   // Secure signing key requirement for HS256 (Must be at least 256 bits)
   private static final SecretKey SIGNING_KEY = Keys.hmacShaKeyFor(
"mySecretKeyMustBeAtLeast256BitsLongToSecureHS256Tokens!".getBytes(StandardCharsets.UTF_8)
   );

   @GetMapping("/authenticate")
   public ResponseEntity<Map<String, String>> authenticate(
         @RequestHeader(value = "Authorization", required = false) String authHeader) {

      LOGGER.info("--- START: Entering authenticate() ---");

      // Validate if the Authorization header is present and format is Basic
      if (authHeader == null || !authHeader.startsWith("Basic ")) {
         LOGGER.error("Authorization header missing or invalid format.");
         return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
```

```
            .body(Collections.singletonMap("error", "Unauthorized"));
    }

    try {
       // Step 2: Read Authorization header and decode the username and password
       String base64Credentials = authHeader.substring("Basic ".length()).trim();
       byte[] decodedBytes = Base64.getDecoder().decode(base64Credentials);
       String credentials = new String(decodedBytes, StandardCharsets.UTF_8);

       // credentials format is "username:password"
       String[] values = credentials.split(":", 2);
       if (values.length != 2) {
          return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(Collections.singletonMap("error", "Invalid header format"));
       }

       String username = values[0];
       String password = values[1];

       LOGGER.info("Attempting authentication for user: {}", username);

       // Match against credential target "user" and "pwd"
       if ("user".equals(username) && "pwd".equals(password)) {

          // Step 3: Generate token based on the user retrieved
          String token = generateJwtToken(username);
```

```
            LOGGER.info("--- END: Exiting authenticate() - Token Generated ---");
            return ResponseEntity.ok(Collections.singletonMap("token", token));
        } else {
            LOGGER.warn("Authentication failed for user: {}", username);
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                    .body(Collections.singletonMap("error", "Unauthorized"));
        }

    } catch (IllegalArgumentException e) {
        LOGGER.error("Failed to decode Base64 credentials", e);
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(Collections.singletonMap("error", "Malformed Base64 payload"));
    }
}

private String generateJwtToken(String username) {
    long nowMillis = System.currentTimeMillis();
    Date now = new Date(nowMillis);

    // Expiration time set to 15 minutes (900,000 ms)
    Date expiryDate = new Date(nowMillis + 900000);

    return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(now)
```

```
            .setExpiration(expiryDate)
            .signWith(SIGNING_KEY, SignatureAlgorithm.HS256)
            .compact();
    }
}
```

Output:

cmd:

curl -s -u user:pwd http://localhost:8090/authenticate